KALINGA INSTITUTE OF INDUSTRIAL TECHNOLOGY (KIIT)

(Deemed to be University)

SCHOOL OF COMPUTER APPLICATIONS

SPRING SEMESTER 2025-26

Date: 12.02.2026

1. **Course Code: BCA 3004**
2. **Course title: Information to Data Science (IDS) Lab** **Marks = 4**

Assignment 5 (Classifcaton N Clustering)

Title of the assignment:

1. Import IRIS dataset.
2. Find out the accuracies of the following Classification algorithms:
    a. Naïve Bayes classifiers
    b. K-Nearest Neighbor (K-NN)
3. Find out the accuracies of the following Clustering algorithm:
    a. K Means

1. **Classification Algorithms**
    a. **Naïve Bayes classifiers**

Advantages of Naïve Bayes Classifier:

- Naïve Bayes is one of the fast and easy ML algorithms to predict a class of datasets.
- It can be used for Binary as well as Multi-class Classifications.
- It performs well in Multi-class predictions as compared to the other Algorithms.
- It is the most popular choice for **text classification problems**.

Disadvantages of Naïve Bayes Classifier:

- Naive Bayes assumes that all features are independent or unrelated, so it cannot learn the relationship between features.

Types of Naïve Bayes Model:

There are three types of Naive Bayes Model, which are given below:

- **Gaussian**: The Gaussian model assumes that features follow a normal distribution. This means if predictors take continuous values instead of discrete, then the model assumes that these values are sampled from the Gaussian distribution.
- **Multinomial**: The Multinomial Naïve Bayes classifier is used when the data is multinomial distributed. It is primarily used for document classification problems, it means a particular document belongs to which category such as Sports, Politics, education, etc.
  The classifier uses the frequency of words for the predictors.
- **Bernoulli**: The Bernoulli classifier works similar to the Multinomial classifier, but the predictor variables are the independent Booleans variables. Such as if a particular word is present or not in a document. This model is also famous for document classification tasks.

Python Implementation of the Naïve Bayes algorithm:

```
# load the iris dataset
from sklearn.datasets import load_iris
iris = load_iris()
# store the feature matrix (X) and response vector (y)
X = iris.data
y = iris.target
# splitting X and y into training and testing sets
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y,  test_size=0.4, random_state=1)
# training the model on training set
from sklearn.naive_bayes import GaussianNB
gnb = GaussianNB()
gnb.fit(X_train, y_train)
# making predictions on the testing set
y_pred = gnb.predict(X_test)
# comparing actual response values (y_test) with predicted response  values (y_pred)
from sklearn import metrics
print("Gaussian Naive Bayes model accuracy(in %):",
metrics.accuracy_score(y_test, y_pred)*100)
```

```
Gaussian Naive Bayes model accuracy(in %): 95.0
```

b. K-Nearest Neighbor (K-NN)

The KNN calculating the distance of point X from all the points. It then finds K (=3) nearest points with least distance to point X.

In the example shown below following steps are performed:

Step 1. Import scikit-learn package.
Step 2. It creates the feature variables and the target variables.
Step 3. Separate the data into the test data and the training data.
Step 4. Generate a k-NN model using neighbors value.
Step 5. Train the model using the data or adjust the model based on the data.
Step 6. Print the prediction values.

Python's Scikit used to implement the KNN algorithm

```
# Import necessary modules
from sklearn.neighbors import KNeighborsClassifier
from sklearn.model_selection import train_test_split
from sklearn.datasets import load_iris
# Loading data
irisData = load_iris()
# Create feature and target arrays
X = irisData.data
y = irisData.target
# Split into training and test set
X_train, X_test, y_train, y_test = train_test_split(
X, y, test_size = 0.2, random_state=42)
knn = KNeighborsClassifier(n_neighbors=7)
knn.fit(X_train, y_train)
# Predict on dataset which model has not seen before
print(knn.predict(X_test))
```

```
[1 0 2 1 1 0 1 2 2 1 2 0 0 0 0 1 2 1 1 2 0 2 0 2 2 2 2 2 0 0]
```

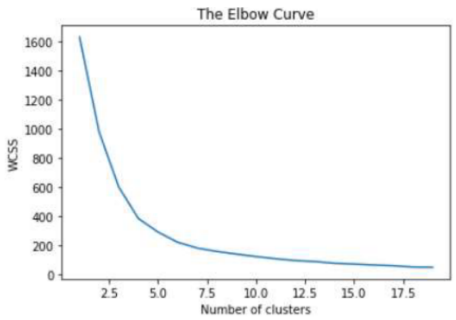
2. Clustering Algorithms

a. K-Mean algorithm

```
# K-MEANS CLUSTERING
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans
x1=10*np.random.rand(100,2)
x1.shape
kmean=KMeans(n_clusters=3)
kmean.fit(x1)
kmean.cluster_centers_
kmean.labels_
wcss = []
for i in range(1,20):
 kmeans = KMeans(n_clusters=i,init='k-means++',max_iter=300,n_init=10,random_state=0)
 kmeans.fit(x1)
 wcss.append(kmeans.inertia_)
 print('Cluster', i, 'Inertia', kmeans.inertia_)
plt.plot(range(1,20),wcss)
plt.title('The Elbow Curve')
plt.xlabel('Number of clusters')
plt.ylabel('WCSS') ##WCSS stands for total within-cluster sum of square
plt.show()
```

```
plt.ylabel('WCSS') ##WCSS stands for total within-cluster sum of square
plt.show()

Cluster 1 Inertia 1633.4729386366648
Cluster 2 Inertia 980.7251659226347
Cluster 3 Inertia 602.6606235675433
Cluster 4 Inertia 386.8266954126674
Cluster 5 Inertia 293.52522127511526
Cluster 6 Inertia 223.57829467348864
Cluster 7 Inertia 183.55842327586788
Cluster 8 Inertia 160.52571627742793
Cluster 9 Inertia 142.12347605903437
Cluster 10 Inertia 125.1405682359277
Cluster 11 Inertia 110.07801216378505
Cluster 12 Inertia 97.71840448709966
Cluster 13 Inertia 91.0468655863335
Cluster 14 Inertia 78.97168184469689
Cluster 15 Inertia 73.78690617997606
Cluster 16 Inertia 67.33331767461436
Cluster 17 Inertia 62.26772230658193
Cluster 18 Inertia 53.44137427335066
Cluster 19 Inertia 51.55574132217379
```



Viva Questions:

1. Differences between Classification and Clustering algorithms.